



STUDENT

0004-AGM

TENTAMEN

230109_DV2606_2110_Salstentamen

Kurskod	--
Bedömningsform	--
Starttid	09.01.2023 14:00
Sluttid	09.01.2023 19:00
Bedömningsfrist	--
PDF skapad	21.05.2024 13:13
Skapad av	Zandra Johansson

- 1 Describe the main characteristics of *single instruction stream, multiple data stream* (SIMD) and *multiple instruction stream, multiple data stream* (MIMD) parallel computers, and also point out some fundamental differences between them.

Further, present some advantage and disadvantage for each type of computer.

Finally, give some example of applications (with motivation) where SIMD and MIMD computers would be preferable over the other, respectively.

On SIMD computers, all processors execute the same instruction at a given time, but each can load their own data by using local indices. This is how GPUs work, all the threads execute the same instruction but we calculate the threads data using the threadId and blockIdx so that we can achieve data parallelism. because of this, we must be considerant of thread divergence in order to utilize the hardware as best as possible in order not to get a bunch of idling where threads are sitting and waiting for other threads to finish executing the instruction.

an advantage of this is that it's very easy for programmers to understand this model.

On MIMD computers, the processors execute different instructions so there's no divergence issue in that sense. All the threads can continue working as long as we don't explicitly stop them using synchronization mechanisms such as a barrier. This means that if we have a loop, one thread may be at the end of the loop and the other has just started. Most modern CPUs use the MIMD model and algorithms using for example pthreads is often built on this mental model. this model can be difficult to graps and understand what's happening where, which is why many programs are written as SPMD - single program multiple data - which means that we write a single program and essentially let the compiler handle a lot of the stuff for us.

Ord: 241

Besvarad.

- 2 Describe (define) what we mean with *speedup* and *efficiency*, in the context of a parallel computer system.

Speedup is the ratio between the sequential and parallel execution times, e.g. if the sequential version finish execution in 100 ms and the parallel in 10 ms, we'd have a speedup of $100 / 10 = 10$.

When measuring speedup and want an accurate representation, its important to take into account the serial implementation so that we're not comparing a sub-optimal sequential version to an optimized parallel version.

Efficiency is how much time the parallel version spends executing excess instructions. We might have a parallel version with good speedup but that executes a lot more instructions which means that algorithm is work inefficient and wastes resources. this can become a problem when we have limited resources which can eat up all the system's resources from other tasks.

Ideally, we'd aim for a speedup of p (or more in some cases where superlinear speedup is achievable) and efficiency ≤ 1 for p processors. This means we're using the hardware effectively without much communication overhead and idling etc.

Ord: 166

Besvarad.

- 3 When we parallelize a sequential application, e.g., by using threads or message-passing, so it can can execute in parallel on N processors, we often find that the parallel program is not N times faster. Describe two important reasons for that.

1. Communication overhead

In parallel programs, processors often needs to communicate with eachother to pass data between eachother, this may in some cases require synchronization mechanisms, atomic operations etc (on shared memory machines) or network latency when sending the data through messages (on distributed memory machines).

2. Idling

If we haven't decomposed our tasks well, we could encounter load imbalancing when some threads / processors are idling while waiting for others to finish their work. this wastes resources and takes time since the processor can still be scheduled even if its not doing anything. to fix this we'd want to decompose our tasks into tasks which takes the same time to execute, so that the idling time is minimal.

Ord: 119

Besvarad.

- 4 Sometimes we find that a system or application has a *superlinear speedup*. What does that mean? Describe two situations when it may occur.

Superlinear speedup is when the speedup is higher than the number of processors, e.g. we get a speedup of 10 when we only use 4 processors. this is often times a warning sign that something's wrong but can occur, particularly in two cases:

1. Lower average memory access times

since we use more processors, we'd have more cache space (assuming local caches) which means we'll have a higher cache hit rate and thus lower average memory access times. think of if we split an array into chunks and each processor only operates on that chunk, that chunk may fit into the local cache of that processor which will result in a lot of cache hits. if we would only have used a single processor and thus only a single cache, the array wouldn't fit in it so we'd get cache misses and having to swap out the data from the cache.

2. Less work per processor

In tree problems, it's often the case that each processor does a significant amount of less work in relation to the number of processors. Say for example if we're looking for a number in a tree, each processor would take one subtree and we could find the thing we're looking for in just a few iterations. a sequential version with a single processor would need to search the tree from left to right which could be the entire tree if we're unlucky.

Ord: 238

Besvarad.

- 5 Shared-memory multiprocessors exist in two different variants: UMA (Uniform Memory Access) and NUMA (Non-uniform Memory Access) multiprocessors. Describe the primary difference between them, and also give an example when one is preferable over the other.

UMA is when each processor have equal access time to any memory location - we have a single big chunk of memory so every processor would need to go out on the bus and find the memory address we want.

On NUMA machines, each processor has its own chunk of memory which would result in longer access times for processors not belonging to that "node".

The primary thing to consider between them is that NUMA machines require locality in order to be efficient, say for example if each processor operates on different parts of the memory, the memory will stay local for a long time resulting in shorter access times. But if we dont have locality the memory would constantly be replaced and we'd get a lot of latencies when we swap pages in and out between the processors.

Ord: 139

Besvarad.

- 6 **Cache coherence** is an important issue to handle in a shared-memory multiprocessor. Describe, by giving an example, why ensuring cache coherence is important. (1p)

Two main approaches exist to maintain cache coherence: *write-invalidate* and *write-update*. Briefly describe each of them, e.g., by drawing a simple picture. (2p)

Describe some advantage and disadvantage with each of the approaches. (1p)

Cache coherence is important because we want to always read the last written value. if the cache wouldn't be coherent a simple program where processor 1 updates a value which processor 2 has cached, and then when processor 2 reads that value it will get the old value from the cache. we want a coherent cache so that processor 2 would get the updated value that processor 1 wrote.

Write invalidate means that whenever we write to a memory address, all the copies of that memory in the caches are invalidated - marked dirty - "old". When a processor would then read that value it would get a cache miss and have to load it back in to get the latest written value.

Write Update means that whenever we write to a memory we also update all the cached copies of that memory location which means that following reads will still hit the cache without having to load it back in.

The advantage of the invalidate protocol is that it's fast and doesn't occupy any write traffic on the memory bus. This is particularly useful when the data is read with long intervals (thus that memory is likely to be replaced from the cache before its read again) where the update protocol would cause overhead due to it's high write traffic on the bus.

On the other side, the invalidate protocol can cause a lot of cache misses if the algorithm relies on locality and memory addresses are read with short intervals. Then it is beneficial for the cache to be updated automatically so that we get a higher cache hit rate.

Ord: 272

Besvarad.

- 7 When writing a parallel application, we can use mainly two approaches: *task (a.k.a. functional) parallelism* and *data parallelism*.
- Describe the main characteristics of each of the two approaches.
 - Further, describe some advantage and disadvantage with each of them.
 - Finally, give some examples of applications (with motivation) that are suitable for task parallelism and data parallelism, respectively.

functional parallelism is when we take our program and divide it into tasks that can be executed in parallel. an example of this is the master-slave pattern where we have a master node which assigns tasks that the processors would do, or the client-server pattern where different processors have different purpose and the client nodes sends requests to the server which handles the incoming requests before sending back the response to the clients. the different slave / clients just does their work without caring about others.

data parallelism is when we have a large data set that we'd like to process in parallel. we would then split the data up into chunks in a smart way (using different data distribution techniques) that we can process in parallel with as little communication overhead as possible between different processors.

this is useful for a lot of algorithms such as sorting algorithms, calculation algorithms, science applications where we get a massive amount of data which a single processor wouldn't handle very well due to processing and memory limitations.

Ord: 175

Besvarad.

8 Consider the sequential C code below for performing matrix-vector multiplication.

```

1  #define SIZE 4096
2  static double a[SIZE][SIZE], x[SIZE], y[SIZE];
3
4  static void init_values(void)
5  {
6      ... // init matrix a and vector x
7  }
8
9  static void matvec_seq(void)
10 {
11     int i, j;
12     for (i = 0; i < SIZE; i++) {
13         y[i] = 0.0;
14         for (j = 0; j < SIZE; j++)
15             y[i] = y[i] + a[i][j] * x[j];
16     }
17 }
18
19 int main(int argc, char **argv)
20 {
21     init_values();
22     matvec_seq();
23 }

```

Write (in C-like pseudo code) a parallel version of the matrix-vector multiplication algorithm, i.e., the function *matvec_seq()*, using a *1-dimensional partitioning*. Clearly state which assumptions you do about the type of machine and programming model. Assume that the number of processors is significantly fewer than SIZE. (2p)

Analyze your solution from a performance point of view, e.g., approximate speed-up, load-balancing, etc. (2p)

```
#DEFINE N_THREADS <some_number>
```

```

void *matvec_prl(void *args) {
    params *args = (params*)args;
    // blockwise partitioning
    int first_row = args->size / args->n_threads * args->tid;
    int last_row = args->size / args->n_threads * (args->tid + 1);

    // each thread does a contiguous chunk of the matrix-vector
    // the rows won't overlap so no synchronization needed, we also dont need to wait for other
    rows before continuing => no dependencies between rows
    for (int row = first_row; row < last_row; row++) {
        y[row] = 0.0;
        for (int col = 0; col < args->size; col++) {
            y[row] = y[row] + a[row][col] * x[col];
        }
    }
}

int main() {

```

```
init_values();
pthread *threads = malloc(N_THREADS * sizeof pthread);
for (int tid = 0; tid < N_THREADS; tid++) {
    params *args = { tid, a, SIZE, N_THREADS };
    pthread_create(&threads[tid], matvec_prl, (void*)args);
}
for (int tid = 0; tid < N_THREADS; tid++) {
    pthread_join(threads[tid]);
}
}
```

for matrix-vector multiplication, we wont run into load-imbancing since the rows are of fixed size and we always traverse the full row (compared to gaussian elimination where the rows get shorter and shorter). we also don't have any synchronization mechanisms, since the rows are not dependent on one another. so i'd assume the algorithm would get approximately p speedup if p is the amount of threads available on the system.

using more threads than available shouldn't make a difference here so if our system has 8 cores / 16 threads we should be good to set the N_THREADS to 16 and expect roughly 16x speedup.

Ord: 252

Besvarad.

9 Thread Blocks and Warps

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

- ☐ Threads in the same Warp can execute on different Streaming Multiprocessors.
- ☐ Threads from different Thread Blocks can execute in the same Warp.
- ☐ A Thread Block cannot be larger than a Warp.
- ☒ Threads in the same Warp cannot execute different instructions at the same time. 
- ☒ A Thread Block can contain many Warps. 

Rätt. 2 av 2 poäng.

10

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

- ☐ A CPU normally has more processor cores than a GPU
- ☒ Context switching on a GPU is normally faster than context switching on a CPU. 
- ☐ The processor cores in a CPU normally executes in SIMD mode.
- ☐ The processor cores on a CPU are normally organized in streaming multiprocessors.
- ☒ A CPU normally has larger caches than a GPU 

Rätt. 2 av 2 poäng.

11

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

- ☐ The CPU can access variables in GPU shared memory
- ☐ All the threads in a thread block see the same registers
- ☒ Threads in different thread blocks cannot access the same variable in GPU shared memory 
- ☒ The content of the CUDA global memory is not changed between two kernel launches in the same program 
- ☐ Accesses to GPU global memory are faster than accesses to GPU shared memory.

Rätt. 2 av 2 poäng.

12

Explain what thread granularity coarsening is. Give an example when thread granularity coarsening can make a program run faster. Also, explain why thread granularity coarsening could make our program run slower.

thread coarsening is when we let a single thread do more work than "normal CUDA applications" (where we often have each thread do one element of the data set)

an example of this could be when we iterate a 3D matrix and can reuse a lot of the memory location calculations for a thread i.e. when we can reuse the column index and do multiple iterations there for example and save instructions as well as registers.

thread coarsening can also be used in conjunction with loop unrolling, say that each thread handles 8 items, we could use loop unrolling on that part and do the addition/multiplication etc inline without needing to calculate memory addresses, handle branching etc which loops require.

thread coarsening could cause our program to run slower if our data is relatively small and we have a lot of resources, and we have enough threads to handle the entire data set. then thread coarsening would cause less parallelism and slower execution time.

Ord: 164

Besvarad.

13

To make our program run faster we would like to overlap communication and computation. In the CUDA part of the course we discussed scenarios when such overlapping was possible. Describe one scenario when we can improve performance by overlapping computation and communication in a CUDA program.

One scenario where we'd like to overlap communication and computation is when we're on a distributed machine with multiple processors and gpu units. then we could use CUDA together with MPI in order to split the dataset onto multiple units.

The CPU hosts would then send the data between eachother, and at the same time we're doing computation on the GPUs. the data chunks could have dependencies with eachother, for examle in the case of 3d convolution where the dataset overlapped and chunk1 needed part of chunk2's data. if we couldn't overlap communication and computatoin here we would have a lot of idling compute time which would slow down our application and reduce performance.

Ord: 114

Besvarad.

14 Privatization

Privatization is a technique that can be used for improving the performance of some CUDA programs, e.g., programs that perform parallel histogramming. Explain how privatization works and why privatization can improve performance.

Privatization means that each thread block (or individual threads on a register level if our data is small enough) would load a local copy of, for example the histogram, into shared memory. the threads in that block would then use that local copy for the computations before we finally sum everything together into a single result.

this is useful in the case where we need atomic operations. atomic operations are very costly on global memory due to the high latencies that comes with read+writing (roughly 1000 cycles) to it - and during this time no other thread would be able to access that memory address. this would slow down our program massively and any performance gains from the computations would be lost in atomic operations and sequential memory accesses.

instead, we'd like to read in local copies of the histogram (or whatever we're doing) and doing the atomic operations on the copy which would be in shared memory which is magnitudes faster. the memory references would still be sequential, but with much lower latencies which will increase performance. finally, we'd sum everything up using global atomic operations - which will still be very slow, but we're doing it a lot less times.

Ord: 202

Besvarad.

15

For sparse matrix computation we discussed two optimizations: 1) sorting the rows so that the row with the largest number of non-zero elements comes first and the row with the smallest number of non-zero elements comes last, 2) transposing the matrix. Explain why and how each of these two optimizations can improve the performance of a CUDA program.

Sorting rows:

This improves performance since we're reducing the thread divergence. Since the rows are sorted by length, adjacent threads i.e. threads in the same warp, would get equal-ish amount of work - reducing thread divergence. this means that warps high up takes long time to finish and warps far down (with short rows) takes a short time to finish, but the threads within the warps will take roughly the same time meaning we won't have much idling within warps.

Transposing the matrix:

This improves performance due to memory bursting - we're reading memory coalesced. When warps launch, the threads will read adjacent memory locations, for example

iteration 1: t0 will read (0,0), t1 reads (0,1), t2 (0,2) and so on.

iteration 2: t0 reads (1,0), t1 reads (1,1) t2 (1,2) ...

this means we benefit from memory bursting thus improving performance

we can also combine these two methods to get even better performance

Ord: 153

Besvarad.

16 Scatter to Gather

Scatter to gather conversion is a technique that can be used for improving the performance of some CUDA program. Explain how scatter to gather conversion works and why this technique can improve performance.

scatter means that each thread reads one input value and writes to a lot of output values

- this is bad cause we'll need synchronization on the write operations to avoid race conditions with for example atomic operations which is bad for performance and will cause a lot of memory latencies.

gather means that each thread reads a lot of the input values and writes to a single output value.

- this is a lot better because the write operations wouldn't need to be synchronized using atomic operations. we wouldn't need synchronization on the reads either since they are reads and doesn't cause race conditions.

so when we do a scatter to gather transformation we alter the algorithm so that we can use the gather pattern instead of the scatter. this can be done using several methods on a case to case basis, but some examples are loop fission to be able to do loop interchange for example. that means that we may have a nested loop where the inner loop is the write index and we want to be able to swap the loops around so that the outer loop is the write index - then we can launch the outer loop using CUDA threads and each thread would be assigned a single output value.

Ord: 216

Besvarad.

17 Tiling

Tiling is a technique that can be used for improving the performance of some CUDA programs, e.g., programs that perform parallel matrix multiplication or convolution. Explain how tiling works and why and how we may need to use `__syncthreads()` when using tiling.

tiling is when we launch threads using a grid and let a block handle a "tile" of the matrix. we would begin with allocating some shared memory for the tile which we'll use as we compute to reduce global memory accesses. here, we need to use `__syncthreads()` to ensure that all threads within the block has loaded their element into shared memory before we start any computation. then, once synchronized, the threads would do the computation part and store the values into the shared memory. when we're done (we might need a second `syncthreads` here depending on the algorithm), we write the result from the shared memory to global memory. this way we have saved a bunch of global memory reads in exchange for the faster shared memory.

when we use tiling, we can also take advantage of prefetching which means we load in the next tile as we compute the current tile. here we'd also need to use `syncthreads` so that we have read in all the necessary data before starting the computation.

Ord: 174

Besvarad.